Day 109 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 16, Day 5 reference

Watch: Gradient Boost Part 1 (of 4): Regression Main Ideas (StatQuest) | Gradient Boost Part 2 (of 4): Regression Details (StatQuest)

`day109_gradient_boosting.py`, segment by segment, in the order `main()` uses things. Trees and boosting are NumPy; scikit-learn's boosting is used only as an outside check. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: F = the model's prediction so far · r = what is still wrong · ν = learning rate · p = probability · h = curvature p(1 – p)

Segment 0 — Setup

Imports and settings

CODE

```
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
with contextlib.redirect_stdout(io.StringIO()):
    import day107_random_forests as d107
d106, d105 = d107.d106, d107.d105
stratified_split = d105.stratified_split
LR = 0.1
DEPTH = 3
ROUNDS = [0, 1, 2, 5, 10, 25, 50, 100, 200, 500]
RATES = [1.0, 0.3, 0.1, 0.03, 0.01]
MAX_ROUNDS = 1500
DEPTHS = [1, 2, 3, 4, 6, 8]
SHOW = [1, 5, 25, 100, 400]
SPLITS = 10
```

- `d106, d105 = d107.d106, d107.d105` — the earlier lessons, through Day 107's imports.
- `RATES = [1.0, 0.3, 0.1, 0.03, 0.01]` — each about a third of the last, which covers the range with few values.
- `MAX_ROUNDS = 1500` — the ceiling; the output shows that no run reached it.

Segment 1 — The regression tree

reg_split: every threshold of a feature at once

MATH

$$gain = S_L^2/n_L + S_R^2/n_R - S^2/n, \quad S = sum\ of\ r\ on\ a\ side$$

CODE

```
def reg_split(X, r, min_leaf=1):
    n, best = len(r), None
    total = r.sum(); base = total ** 2 / n
    for f in range(X.shape[1]):
        o = np.argsort(X[:, f], kind="stable"); xs, rs = X[o, f], r[o]
        sl = np.cumsum(rs)[:-1]; nl = np.arange(1, n)
        ok = (xs[1:] != xs[:-1]) & (nl >= min_leaf) & (n - nl >= min_leaf)
        if not ok.any():
            continue
        gain = np.where(ok, sl ** 2 / nl + (total - sl) ** 2 / (n - nl) - base, -np.inf)
        i = int(np.argmax(gain))
        if best is None or gain[i] > best[2] + 1e-12:
            best = (f, (xs[i] + xs[i + 1]) / 2, float(gain[i]))
    return best if best and best[2] > 1e-12 else None
```

- `sl = np.cumsum(rs)[:-1]` — running sum of targets: the left side's total at every gap.
- `sl ** 2 / nl + (total - sl) ** 2 / (n - nl) - base` — the formula above, for all gaps at once; (total - sl) is the right side.
- `np.where(ok, ..., -np.inf)` — gaps between equal values can never win.

grow_reg, set_leaves and predict_reg

CODE

```
def grow_reg(X, r, idx, max_depth, min_leaf=1, depth=0):
    node = {"idx": idx}
    if depth >= max_depth or len(idx) < 2:
        return node
    s = reg_split(X[idx], r[idx], min_leaf)
    if s is None:
        return node
    m = X[idx, s[0]] <= s[1]
    node.update(feature=s[0], threshold=s[1], left=grow_reg(X, r, idx[m], max_depth, min_leaf,
        depth + 1),
                right=grow_reg(X, r, idx[~m], max_depth, min_leaf, depth + 1))
    return node
def set_leaves(node, value):
    if "feature" not in node:
        node["value"] = value(node["idx"]); return
    set_leaves(node["left"], value); set_leaves(node["right"], value)
def predict_reg(node, X):
    out = np.zeros(len(X)); stack = [(node, np.arange(len(X)))]
    while stack:
        n, idx = stack.pop()
        if "feature" not in n:
            out[idx] = n["value"]
            continue
        m = X[idx, n["feature"]] <= n["threshold"]
        stack.append((n["left"], idx[m])); stack.append((n["right"], idx[~m]))
    return out
```

- `node = {"idx": idx}` — a leaf remembers its rows, because its value depends on the loss.
- `set_leaves(node, value)` — value is a function: rows in, one number out.
- `node["value"] = value(node["idx"]); return` — two statements on one line, stopping at a leaf.

Segment 2 — Boosting

sigmoid, rmse and logloss

MATH

$$p = 1 / (1 + e^{-F}), \quad L = log(1 + e^{-(2y-1)F})$$

CODE

```
def sigmoid(F):
    return 1 / (1 + np.exp(-F))
def rmse(F, y):
    return float(np.sqrt(np.mean((y - F) ** 2)))
def logloss(F, y):
    return float(np.mean(np.logaddexp(0, -F * (2 * y - 1))))
```

- `np.logaddexp(0, a)` — log(1 + exp(a)) without overflow when a is large.
- `2 * y - 1` — turns labels 0 and 1 into -1 and +1.

fit_boost: the loop

MATH

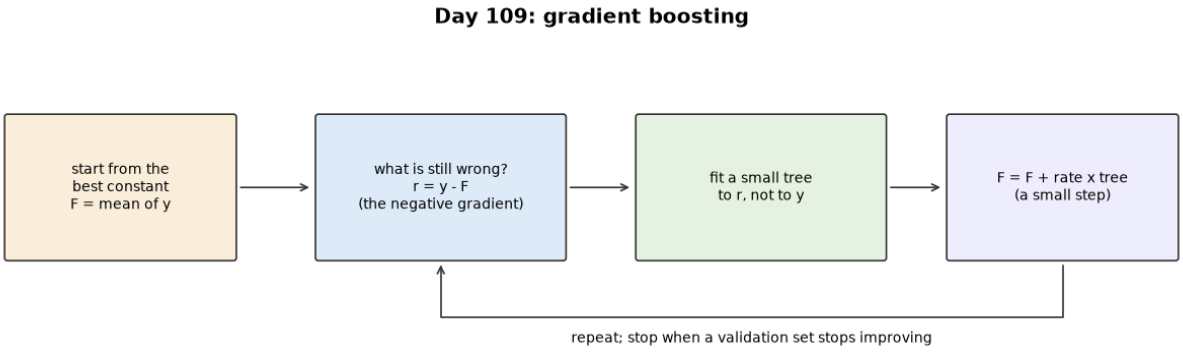
$$F_m = F_{m-1} + \nu\, h_m(x); \quad leaf = \Sigma\, r / \Sigma\, p(1-p) \text{ for log loss}$$

CODE

```
def fit_boost(X, y, loss, rounds, lr, max_depth, evals=(), min_leaf=1):
    y = y.astype(float); idx = np.arange(len(y))
    base = y.mean() if loss == "squared" else np.log(y.mean() / (1 - y.mean()))
    F = np.full(len(y), base); trees = []
    E = [np.zeros((rounds + 1, len(v))) + base for v in evals]
    for m in range(rounds):
        if loss == "squared":
            r = y - F
            leaf = lambda i: r[i].mean()
        else:
            p = sigmoid(F); r = y - p; h = p * (1 - p)
            leaf = lambda i: r[i].sum() / max(h[i].sum(), 1e-12)
        t = grow_reg(X, r, idx, max_depth, min_leaf); set_leaves(t, leaf); trees.append(t)
        F = F + lr * predict_reg(t, X)
        for e, v in zip(E, evals):
            e[m + 1] = e[m] + lr * predict_reg(t, v)
    return {"base": base, "lr": lr, "trees": trees, "train": F}, E
```

- `leaf = lambda i: r[i].mean()` — a function built inside the loop: it uses this round's r.
- `E = [np.zeros((rounds + 1, len(v))) + base for v in evals]` — a table per evaluation set: one row per round, one column per example.
- `e[m + 1] = e[m] + lr * predict_reg(t, v)` — each round's prediction from the last: all stages are kept, with no refitting.
- `for e, v in zip(E, evals):` — walks the two lists together.

SYSTEM DIAGRAM



Gradient boosting.

Segment 3 — The experiments

stop_round and split_rows

CODE

```
def stop_round(E_val, y_val, score):
    return int(np.argmin([score(f, y_val) for f in E_val]))
...
def split_rows(n, frac, seed):
    o = np.random.RandomState(seed).permutation(n); k = int(round(n * frac))
    return o[k:], o[:k]
```

- `for f in E_val` — iterating over a two-dimensional array gives one row at a time: one round.
- `score` — a function passed in, so the same code stops on RMSE or on log loss.
- `o[k:], o[:k]` — a shuffled order, cut in two.

Flipped rows, inside demo_noise

CODE

```
wrong = np.isin(tr, flip)
_, E = fit_boost(X[tr], yn[tr], "logistic", max(keep), LR, DEPTH, evals=(X[tr], X[te]))
for i, k in enumerate(keep):
    r = np.abs(yn[tr] - sigmoid(E[0][k])); A[s, i] = np.mean((E[1][k] > 0) == y[te]); C[s, i] =
        r[~wrong].mean(); W[s, i] = r[wrong].mean()
```

- `np.isin(tr, flip)` — True for training rows whose label was flipped.
- `r[~wrong].mean()` — the residuals of correct rows; r[wrong] gives flipped ones.
- `E[0][k]` — the prediction after k rounds, from the table of stages.

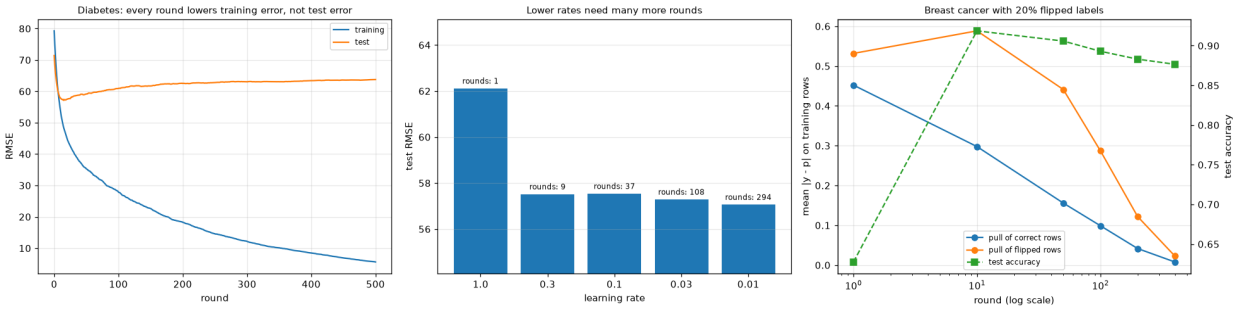
Linear regression, inside demo_compare

CODE

```
r = stop_round(E[0], yd[va], rmse); A = np.column_stack([np.ones(len(a)), Xd[a]])
w = np.linalg.lstsq(A, yd[a], rcond=None)[0]
```

- `np.column_stack([np.ones(len(a)), Xd[a]])` — a column of ones, so that the line has an intercept.
- `np.linalg.lstsq(A, y, rcond=None)[0]` — the weights with the least squared error; [0] takes them from the result.

REAL GRAPH



Real graph from the script: error correction on Diabetes, learning rates, and flipped labels.

Quick Syntax Glossary — new or reinforced constructs

- `np.cumsum(a)` — running totals.
- `np.logaddexp(a, b)` — log(exp(a) + exp(b)), safely.
- `np.isin(a, b)` — which items of a are in b.
- `np.linalg.lstsq(A, y, rcond=None)` — the least-squares solution.
- `lambda i: r[i].mean()` — a one-line function.
- `for a, b in zip(x, y):` — two lists, side by side.
- `"s" if m > 1 else ""` — a conditional expression, here for a plural ending.

End of Day 109 syntax reference. For the experiments and what they showed, see the Day 109 complete guide.